SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	V.RAKSHANA	**Reg. No.:**	192425137
Date:	15.8.2026	**Duration:**	60 minutes

Code of Conduct

I V.Rakshana(192425137)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Rakshana

Problem Statement 24: Smart Museum Artifact Preservation Platform

1.Problem Overview

The Smart Museum Artifact Preservation Platform is a full-stack software system developed to digitally manage museum artifacts and support their preservation. Museums need to maintain accurate information about artifacts, their physical condition, environmental conditions, and preservation activities. Maintaining these records manually or in separate systems can make information difficult to search, update, monitor, and share.

The proposed platform provides a centralized digital system in which artifact information and preservation records can be maintained. It combines artifact management, condition tracking, environmental monitoring, alerts, dashboard visualization, and digital record maintenance in one application.

Objectives

- Create centralized digital records for museum artifacts.
- Allow authorized users to register, search, update, and delete artifact records.
- Track the physical condition of artifacts over time.
- Monitor environmental factors such as temperature and humidity.
- Improve maintainability and collaboration through modular software design, Git, testing, documentation, and containerization.

2. Repository Organization

A modular repository structure improves readability, maintainability, testing, and collaboration. Application logic, database models, routes, frontend files, static resources, and tests should be separated rather than placing the complete application in one file.

Folder/File	Purpose
app.py	Main Flask application and application configuration.
models/	Database models such as the Artifact model.
routes/	Routes for authentication, artifact operations, dashboard, and alerts.
templates/	HTML pages and frontend templates.
static/	CSS, JavaScript, images, and other static resources.
tests/	Unit and integration test files.
database/	Database-related files or database configuration.
docs/	Project documentation and supporting documents.
requirements.txt	Python dependencies required by the application.
Dockerfile	Instructions for building the application container.
.gitignore	Files and folders that should not be committed to Git.
README.md	Project overview, installation, usage, testing, architecture, and deployment information.

This organization provides separation of concerns. Developers can work on different components independently, changes are easier to locate, testing becomes more systematic, and future modifications can be made without affecting unrelated components.

3. Code Quality Review

The code should be reviewed for readability, modularity, coding standards, documentation, validation, exception handling, and maintainability. The recommended implementation should use meaningful names for classes, variables, functions, routes, and database fields.

Strengths

- Meaningful naming makes the purpose of variables, functions, routes, and models easy to understand.
- Modular functions reduce the complexity of individual program units.
- Separation of concerns keeps database, application, route, and presentation logic organized.
- Clear comments and documentation help other developers understand the project.

Areas for Improvement

- Add stronger input validation for artifact and environmental data.
- Use centralized error handling rather than repeating error-handling logic.
- Add structured logging for important application events and errors.
- Use secure password handling and avoid storing sensitive credentials directly in source code.
- Keep configuration values separate from application code.

Overall, a modular Flask application with clear naming, validation, exception handling, documentation, and reusable components would be easier to maintain and extend.

4. Version Control Evaluation

Git and GitHub provide version control for the project. They maintain a history of changes, support collaboration, allow developers to work on separate features, and provide the ability to review or roll back changes.

Branching Strategy

A feature-based branching strategy can be used. The main branch can contain stable code, while separate feature branches can be used for artifact management, dashboard development, preservation alerts, testing, and Docker configuration. Completed features can be reviewed and merged into the main branch.

5. Code Testing and Validation

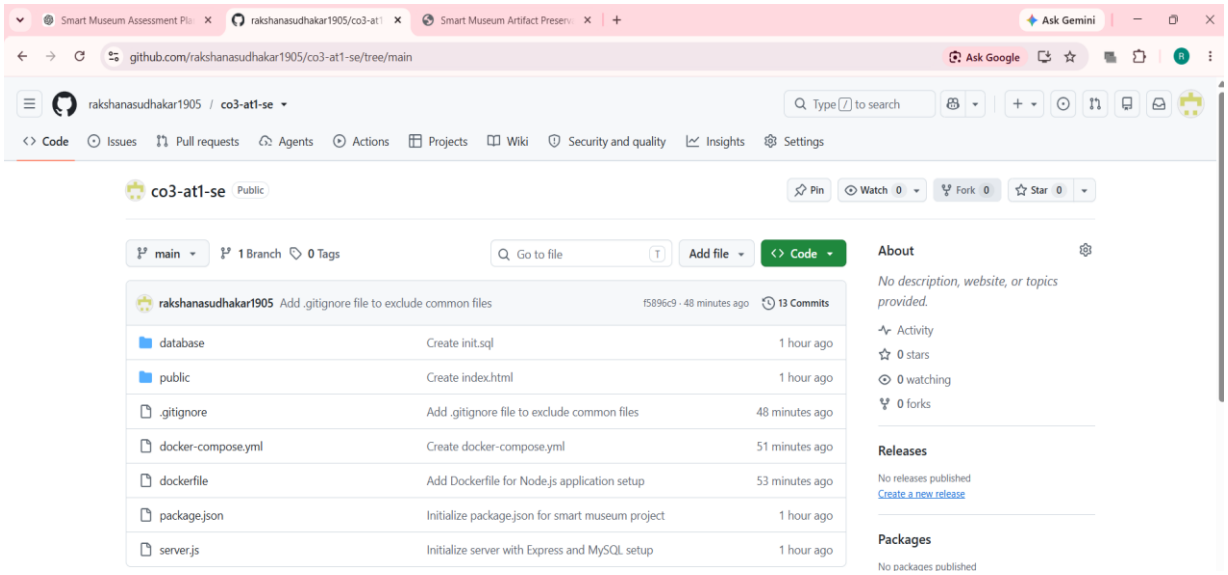

Fig 1: GITHUB Respository

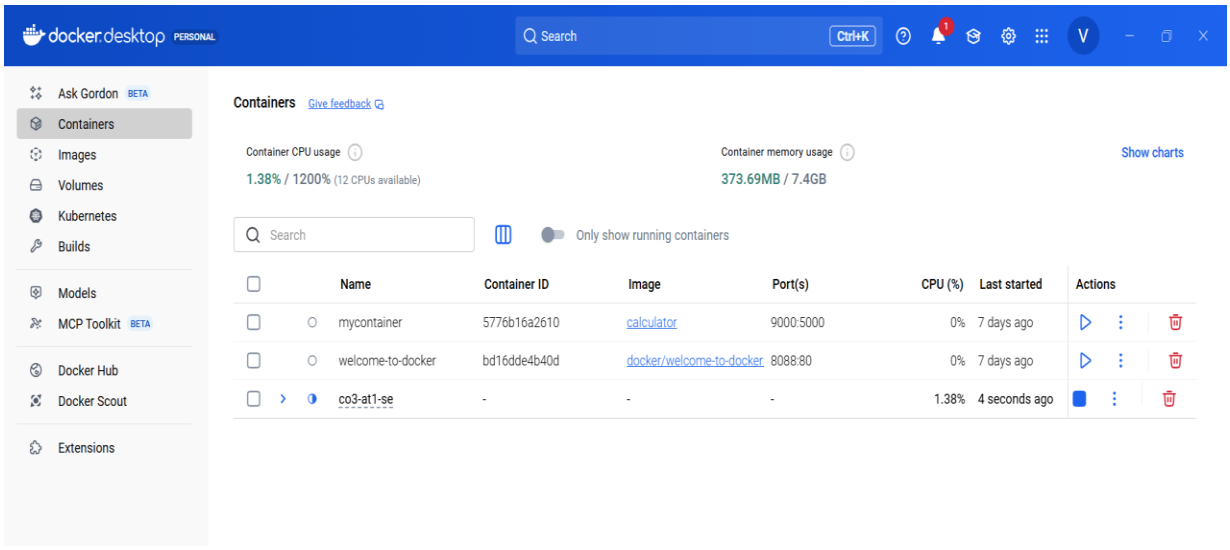

Fig 2: Docker Implementation

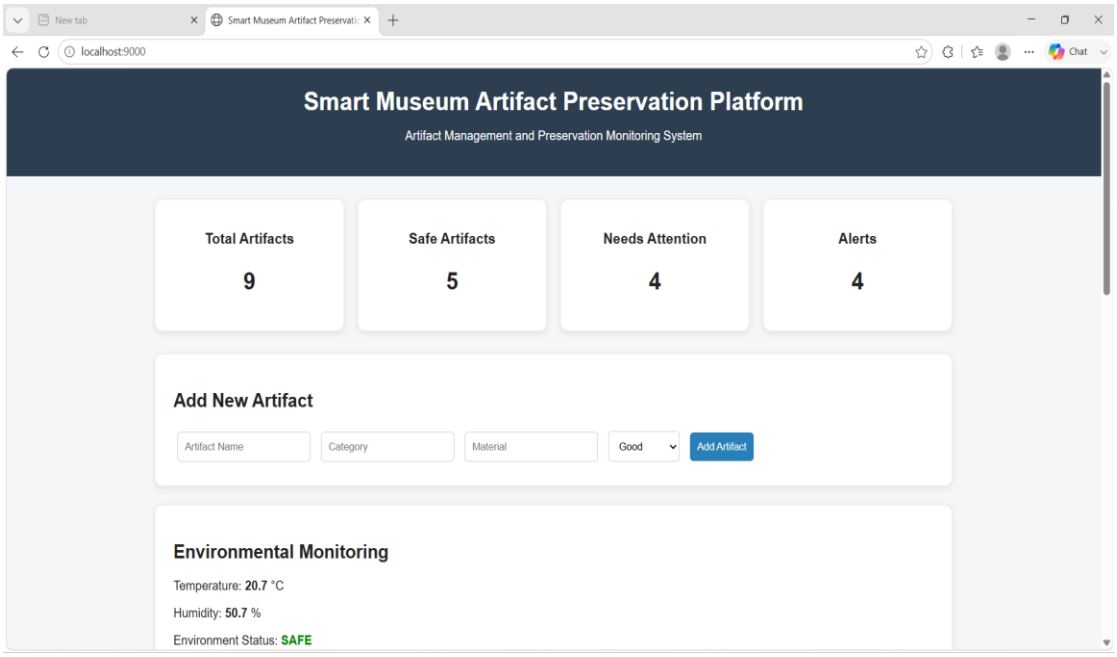

Fig 3a: Dashboard

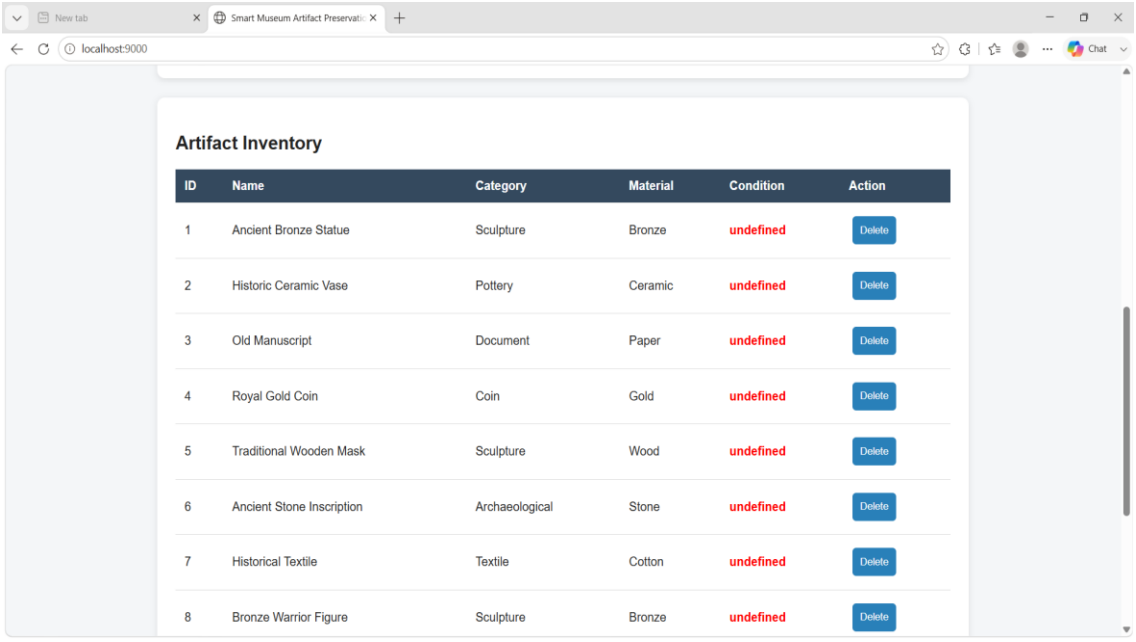

Fig 3b: Dashboard

6. Repository Documentation

The README should be complete enough for another developer to understand and run the project. It should contain the problem statement, objectives, features, technology stack, repository structure, installation procedure, execution steps, database setup, testing instructions, screenshots, Docker/containerization instructions, and future enhancements.

7. Code Review Findings and Recommendations

The overall review indicates that the platform should follow modular software engineering practices and maintain clear separation between presentation, application, and data responsibilities. The repository should contain meaningful Git history, systematic tests, clear documentation, and Docker configuration.

Recommendations

- Improve validation and centralized exception handling.
- Increase unit and integration test coverage.
- Use PostgreSQL for production-scale database deployment.
- Integrate IoT sensors for real-time environmental monitoring.
- Use image-based artifact condition assessment.
- Provide cloud backup for preservation records.
- Implement CI/CD for automated build and testing.

8.Architecture-Oriented Evaluation

System Requirements

Functional requirements include login, artifact registration, artifact search, artifact update, condition tracking, environmental monitoring, preservation alerts, and dashboard visualization. Non-functional requirements include security, usability, reliability, maintainability, scalability, and performance.

Architecture Diagram and Component Design

Layer	Components	Responsibility
Presentation Layer	HTML, CSS, JavaScript, Dashboard	User interaction and visualization
Application Layer	Flask, Authentication, Artifact Management, Monitoring, Alert Management	Business logic, validation, routing, and alert processing
Data Layer	SQLite / SQLAlchemy, Database	Persistent storage and retrieval of artifact and preservation information

Main components are Authentication, Artifact Management, Monitoring, Alert Management, Dashboard, Database, and Testing.

Component Interaction and Quality Attributes

A user interacts with the presentation layer. Requests are sent to Flask routes in the application layer. Business logic validates and processes the request, then communicates with the database layer through

SQLAlchemy. Environmental monitoring data is evaluated by the application logic, and abnormal values can produce preservation alerts. The dashboard retrieves relevant information and presents it to users.

Quality Attribute	How It Is Supported
Security	Authentication, validation, secure configuration, and role-based access control as a future enhancement.
Usability	Clear dashboard, search functions, and understandable error messages.
Reliability	Validation, exception handling, testing, and database persistence.
Maintainability	Modular repository, separation of concerns, documentation, and meaningful naming.
Scalability	Layered architecture and future migration to PostgreSQL/cloud infrastructure.
Performance	Efficient database operations, modular processing, and appropriate indexing as the system grows.

Architectural Justification

The 3-tier architecture is suitable because it separates user interface responsibilities from business logic and data storage. This reduces coupling and allows individual layers to be modified independently. For example, the database can be migrated from SQLite to PostgreSQL without redesigning the complete presentation layer. Similarly, dashboard interfaces can be changed without rewriting database models. This architecture therefore supports maintainability, collaboration, testing, and future scalability.

9.Conclusion

The Smart Museum Artifact Preservation Platform demonstrates how a full-stack application can digitally manage museum artifacts and support preservation activities. A modular repository, meaningful Git history, systematic testing, clear documentation, containerization, and a well-justified software architecture provide evidence of maintainable and collaborative software development. The completed assessment should be supported by actual screenshots and execution results from the student's implemented repository.